# 청담 DB 스키마: 수집·추출·알림 13개 테이블

슬라이드별 상세 발표 대본

| 항목 | 내용 |
|---|---|
| 원본 자료 | 청담 DB 스키마 SCHEMA.md (2026-10-21, 초안) |
| 발표 시간 | 20분 |
| 슬라이드 수 | 12장 |
| 대본 기준 예상 발화 시간 | 약 15.9분 (한국어 300자/분 기준 추정) |
| 청중 | 팀원 A(Python 수집·추출)와 B(Spring 웹·구독) 개발자 |
| 언어 | 슬라이드 ko / 대본 ko |

## 전체 흐름

청담의 DB 스키마는 A와 B가 DB로만 주고받기 위한 계약서입니다. 먼저 쓰는 쪽을 테이블마다 하나로 나눈 원칙과 테이블 전체 지도를 보고, 수집·공고·버전·추출·알림 영역을 순서대로 살펴봅니다. 이어서 정정 공고 하루의 데이터 흐름과 개인정보 파기 규칙을 확인하고, 마지막으로 합의가 필요한 미정 사항 6개를 정리합니다.

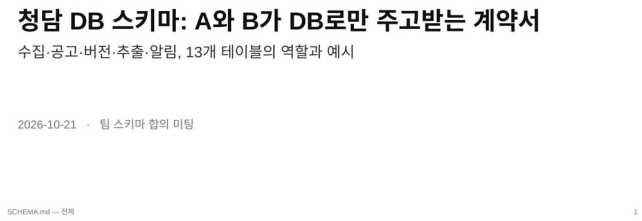
## 슬라이드 목차와 시간 배분

| # | 제목 | 계획(분) | 대본 추정(분) |
|---|---|---|---|
| 1 | 청담 DB 스키마: A와 B가 DB로만 주고받는 계약서 | 0.5 | 0.5 |
| 2 | 스키마는 A와 B가 서로의 코드를 보지 않고 일하게 해 주는 계약서다 | 1.5 | 1.0 |
| 3 | 테이블은 5개 영역, 테이블마다 쓰는 쪽은 한 명이다 | 1.5 | 0.8 |
| 4 | 여섯 가지 설계 원칙이 "무엇을 어디에 저장하는가"를 결정한다 | 2 | 1.5 |
| 5 | crawl_run은 "수집이 살아 있는가"를 알려 주는 신호등이다 | 1 | 1.1 |
| 6 | 공고는 notice 한 행, 일정과 주택형은 자식 테이블로 푼다 | 2.5 | 1.8 |
| 7 | 정정 공고는 덮어쓰지 않고, 한 트랜잭션으로 새 버전을 쌓는다 | 2 | 1.9 |
| 8 | 추출 결과는 작업 → 항목 → 근거 순으로 내려가며 검증 가능하게 남는다 | 2.5 | 2.0 |
| 9 | 알림은 구독 → 적재 → 발송 → 이력으로 흐르고, 두 쪽이 바통을 넘긴다 | 1.5 | 1.3 |
| 10 | 중복 구독, 하루 두 통, 같은 공고 재발송은 제약 조건이 막는다 | 2 | 1.6 |
| 11 | 개인정보는 이메일 하나뿐이며, 해지·미확인·반송 때 규칙대로 지운다 | 1.5 | 1.1 |

| | | | |
|---|---|---|---|
| 12 | 오늘 합의할 6가지: 스키마 소유자, 코드명, 조건 키가 핵심이다 | 1.5 | 1.3 |

# 슬라이드 1. 청담 DB 스키마: A와 B가 DB로만 주고받는 계약서

약 0.5분 · 출처: SCHEMA.md — 전체




**핵심 메시지:** 수집·공고·버전·추출·알림, 13개 테이블의 역할과 예시

### 발표 대본

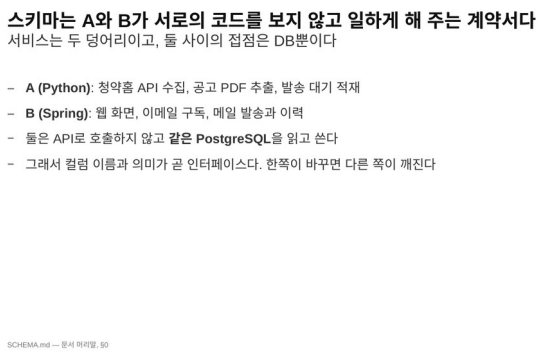
오늘은 청담 DB 스키마를 설명드리겠습니다. 이 문서는 A가 만드는 Python 수집·추출 쪽과, B가 만드는 Spring 웹·구독 쪽이 오직 DB로만 소통하기 위한 계약서입니다. 20분 동안 어떤 테이블이 있고, 각각 무슨 역할이며, 실제 데이터가 어떻게 흘러가는지를 예시와 함께 보겠습니다. 그리고 마지막에는 둘이 합의해야 할 미정 사항을 같이 정하겠습니다.

### 다음 슬라이드로 넘어가는 말

먼저 왜 스키마를 계약서라고 부르는지부터 보겠습니다.

# 슬라이드 2. 스키마는 A와 B가 서로의 코드를 보지 않고 일하게 해 주는 계약서다

약 1.5분 · 출처: SCHEMA.md — 문서 머리말, §0




**핵심 메시지:** 서비스는 두 덩어리이고, 둘 사이의 접점은 DB뿐이다

### 먼저 알아야 할 배경지식

청담은 청약홈 분양 공고를 모아 보여 주고, 공고 PDF에서 청약 요건을 뽑아 요약하고, 조건에 맞는 새 공고와 정정 공고를 이메일로 알려 주는 서비스입니다. 구현은 A가 Python으로 수집과 추출을, B가 Spring으로 웹과 구독을 맡습니다.

### 화면에서 먼저 가리킬 부분

화면 왼쪽의 A와 B 두 항목을 먼저 가리키고, 세 번째 줄의 '같은 PostgreSQL'로 시선을 옮깁니다.

### 발표 대본

먼저 이 스키마가 왜 필요한지부터 말씀드리겠습니다. 우리 서비스는 크게 두 덩어리입니다. A가 맡은 Python 쪽은 청약홈 API에서 공고를 수집하고, PDF에서 청약 요건을 뽑아내고, 보낼 메일을 발송 대기열에 쌓습니다. B가 맡은 Spring 쪽은 웹 화면을 보여 주고, 이메일 구독을 받고, 실제로 메일을 보내고 이력을 남깁니다.

그런데 이 둘은 서로 API를 호출하지 않습니다. 같은 PostgreSQL 하나를 같이 읽고 씁니다. 그러니까 테이블 이름과 컬럼 의미가 곧 두 서비스 사이의 인터페이스가 됩니다. 한쪽이 컬럼 하나를 바꾸면 다른 쪽 코드가 조용히 깨질 수 있습니다. 그래서 이 문서를 계약서라고 부르고, 오늘 미팅에서 이 계약을 같이 확인하려는 겁니다.

### 헷갈리기 쉬운 점

• 문서 상태가 '초안 (A·B 합의 전)'입니다. 오늘 설명하는 내용은 확정이 아니라 합의 대상입니다.

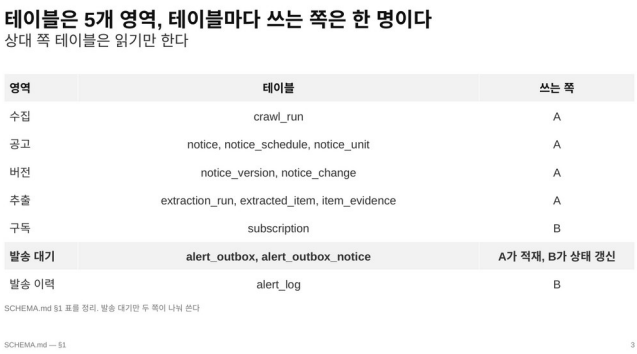
• 문서 곳곳의 P-01, E-05 같은 번호는 기획 문서의 결정 사항과 예외 상황 번호입니다. 이 스키마 문서 안에서 정의되지 않으니 필요하면 해당 기획 문서를 열어야 합니다.

### 다음 슬라이드로 넘어가는 말

그럼 어떤 테이블이 있고 누가 쓰는지 전체 지도를 보겠습니다.

# 슬라이드 3. 테이블은 5개 영역, 테이블마다 쓰는 쪽은 한 명이다

약 1.5분 · 출처: SCHEMA.md — §1



**테이블은 5개 영역, 테이블마다 쓰는 쪽은 한 명이다**
상대 쪽 테이블은 읽기만 한다

| 영역 | 테이블 | 쓰는 쪽 |
|---|---|---|
| 수집 | crawl_run | A |
| 공고 | notice, notice_schedule, notice_unit | A |
| 버전 | notice_version, notice_change | A |
| 추출 | extraction_run, extracted_item, item_evidence | A |
| 구독 | subscription | B |
| **발송 대기** | **alert_outbox, alert_outbox_notice** | **A가 적재, B가 상태 갱신** |
| 발송 이력 | alert_log | B |

SCHEMA.md §1 표를 정리. 발송 대기만 두 쪽이 나눠 쓴다

SCHEMA.md — §1 3

**핵심 메시지:** 상대 쪽 테이블은 읽기만 한다

### 화면에서 먼저 가리킬 부분
표의 맨 오른쪽 '쓰는 쪽' 열을 위에서 아래로 훑고, 음영으로 강조된 발송 대기 행에서 멈춥니다.

### 발표 대본
이게 전체 지도입니다. 테이블은 열세 개이고 다섯 영역으로 묶입니다. 수집, 공고, 버전, 추출, 그리고 알림입니다. 알림은 구독, 발송 대기, 발송 이력으로 나눴습니다.

표에서 오른쪽 열을 보시면 쓰는 쪽이 거의 한 명으로 정해져 있습니다. 수집부터 추출까지는 전부 A가 쓰고, 구독과 발송 이력은 B가 씁니다. 상대 쪽 테이블은 읽기만 합니다. 예외는 음영이 들어간 발송 대기 행 하나입니다. 이건 A가 07시 30분 직전에 보낼 메일을 쌓고, B가 그걸 읽어서 보낸 뒤 상태를 바꾸니까 두 쪽이 나눠 쓰는 구조입니다. 이 이유는 알림 영역에서 다시 설명드리겠습니다.

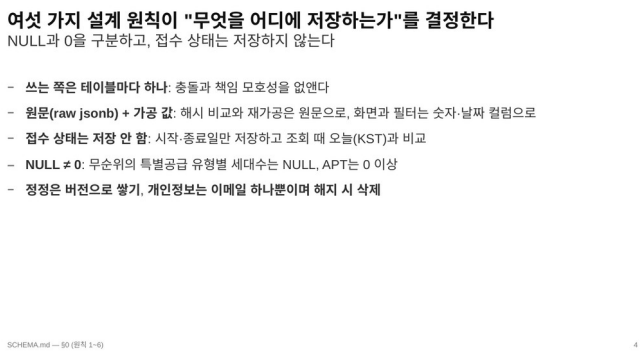
### 축·범례·색상·화살표·박스의 의미
음영 강조 행 하나만 두 쪽이 나눠 쓰는 테이블입니다.

### 숫자로 확인하기
세어 보면 crawl_run 1, 공고 3, 버전 2, 추출 3, 구독 1, 발송 대기 2, 이력 1로 모두 13개입니다.

### 다음 슬라이드로 넘어가는 말
이제 이 구조를 지탱하는 설계 원칙 여섯 가지를 보겠습니다.

# 슬라이드 4. 여섯 가지 설계 원칙이 "무엇을 어디에 저장하는가"를 결정한다

약 2분 · 출처: SCHEMA.md — §0 (원칙 1~6)




**핵심 메시지:** NULL과 0을 구분하고, 접수 상태는 저장하지 않는다

### 먼저 알아야 할 배경지식

청약홈 공고는 공고 후에 접수 일정이나 내용이 정정되는 일이 있고, 무순위 공고는 APT 공고와 달리 특별공급 유형별 세대수 필드가 아예 응답에 없습니다.

### 화면에서 먼저 가리킬 부분

세 번째 줄 '접수 상태는 저장 안 함'과 네 번째 줄 'NULL ≠ 0'에 먼저 밑줄을 긋듯 짚습니다.

### 발표 대본

설계 원칙은 여섯 가지인데, 구현에서 가장 자주 부딪힐 네 가지를 중심으로 말씀드리겠습니다.

첫째, 쓰는 쪽은 테이블마다 하나입니다. 앞에서 본 그대로입니다.

둘째, API 값과 가공 값을 둘 다 둡니다. 청약홈 응답 원문은 raw라는 jsonb 컬럼에 그대로 남깁니다. 그래야 해시로 정정을 감지하고, 나중에 가공 규칙이 바뀌어도 원문에서 다시 만들 수 있습니다. 대신 화면과 필터에 쓰는 값은 날짜와 숫자 컬럼으로 따로 둡니다.

셋째, 접수 상태는 저장하지 않습니다. '접수 중', '마감' 같은 값을 컬럼에 넣으면 날짜가 바뀔 때마다 갱신해야 하고 틀리기 쉽습니다. 시작일과 종료일만 저장하고, 조회할 때 오늘 날짜와 비교해서 계산합니다.

넷째, NULL과 0을 구분합니다. 예를 들어 무순위 공고는 특별공급 유형별 세대수를 API가 주지 않습니다. 이때 0이라고 넣으면 '없다'와 '모른다'가 섞입니다. 그래서 모르면 NULL, APT에서 진짜 0이면 0으로 구분하고, 필터가 NULL인 공고는 제외하거나 안내를 띄우는 기준으로 씁니다.

나머지 두 가지, 정정을 버전으로 쌓는 것과 개인정보를 이메일 하나로 줄이는 것은 해당 영역에서 자세히 보겠습니다.

### 헷갈리기 쉬운 점

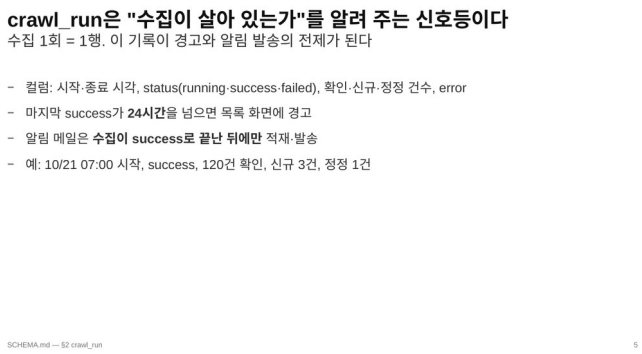
• '접수 중' 같은 상태 컬럼이 없다고 놀라지 마세요. 의도된 설계입니다. 필터는 rcept_start와 rcept_end를 오늘(KST)과 비교하는 쿼리입니다.

• 문서의 rcept_start·rcept_end는 수집 때 계산해서 '저장'합니다. 저장하지 않는 것은 상태(진행 중/마감)이고, 시작·종료일은 저장합니다.

**다음 슬라이드로 넘어가는 말**
원칙을 봤으니 영역별로 들어가겠습니다. 수집부터 공고까지입니다.

# 슬라이드 5. crawl_run은 "수집이 살아 있는가"를 알려 주는 신호등이다

약 1분 · 출처: SCHEMA.md — §2 crawl_run




**핵심 메시지:** 수집 1회 = 1행. 이 기록이 경고와 알림 발송의 전제가 된다

**화면에서 먼저 가리킬 부분**
맨 아래 예시 줄을 먼저 읽고, 위의 컬럼 설명으로 올라갑니다.

**발표 대본**
수집 영역은 테이블이 하나입니다. crawl_run은 수집을 한 번 돌릴 때마다 한 행이 생기는 실행 기록입니다. 시작과 종료 시각, 성공 여부, 이번에 몇 건을 확인했고 몇 건이 새 공고이며 몇 건이 정정이었는지, 실패하면 오류 내용을 남깁니다.

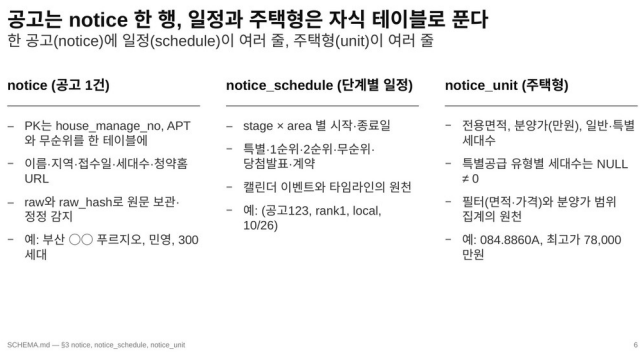
이 테이블이 중요한 이유는 두 가지입니다. 하나는 마지막 성공이 24시간을 넘으면 목록 화면에 데이터가 오래됐다는 경고를 띄우는 근거가 된다는 점입니다. 다른 하나는 알림 메일이 수집이 성공으로 끝난 뒤에만 나가도록 하는 전제라는 점입니다. 수집이 반쯤 실패한 상태에서 '새 공고 없음'으로 메일을 보내면 안 되니까요.

예를 들어 오늘 7시에 시작해서 성공했고, 120건을 확인했고, 새 공고 3건, 정정 1건이었다면 한 행에 그렇게 기록됩니다. 성공 행만 빠르게 찾도록 마지막 성공 시각에 부분 인덱스가 걸려 있습니다.

**다음 슬라이드로 넘어가는 말**
수집이 성공하면 이제 공고 자체를 담는 핵심 테이블들을 보겠습니다.

# 슬라이드 6. 공고는 notice 한 행, 일정과 주택형은 자식 테이블로 푼다

약 2.5분 · 출처: SCHEMA.md — §3 notice, notice_schedule, notice_unit




**핵심 메시지:** 한 공고(notice)에 일정(schedule)이 여러 줄, 주택형(unit)이 여러 줄

### 먼저 알아야 할 배경지식

청약홈 API는 한 공고에 대해 공고 정보 한 건과 주택형(타입)별 정보 여러 건을 따로 줍니다. 접수 일정도 특별공급, 1순위, 2순위처럼 단계별로 컬럼이 따로 있고, 1순위·2순위는 해당 지역, 기타 경기, 기타 지역 접수일이 다릅니다.

### 화면에서 먼저 가리킬 부분

왼쪽 notice 열의 첫 줄 'PK는 house_manage_no'를 먼저 짚고, 오른쪽으로 두 열을 차례로 가리킵니다.

### 발표 대본

공고 영역은 세 개의 테이블이 부모와 자식 관계로 붙어 있습니다.

왼쪽 notice는 공고 한 건을 나타내는 중심 테이블입니다. 기본키는 청약홈의 주택관리번호이고, APT와 무순위 공고의 번호 대역이 겹치지 않아서 한 테이블에 같이 둡니다. 이름, 지역, 접수 시작·종료일, 총 세대수, 청약홈 상세 URL 같은 화면용 값이 컬럼으로 있고, 원문은 raw에, 정정 감지용 해시는 raw_hash에 있습니다. 예를 들면 '부산 ○○ 푸르지오, 민영, 300세대, 접수 10월 25일부터 27일'이 한 행입니다.

가운데 notice_schedule은 단계별 일정입니다. API는 특별공급 접수일, 1순위 해당지역 접수일, 1순위 기타경기 접수일처럼 컬럼이 옆으로 길게 붙어 있는데, 이걸 단계와 지역이라는 두 값을 가진 행으로 풀었습니다. 그래서 캘린더에 이벤트를 만들거나 상세 화면 타임라인을 그릴 때 그냥 행을 순서대로 읽으면 됩니다. 예를 들어 공고 123의 1순위 해당지역은 10월 26일 하루라는 식입니다.

오른쪽 notice_unit은 주택형별 정보입니다. 84형 A타입이라는 단위로 전용면적, 최고 분양가, 일반공급과 특별공급 세대수가 들어 있습니다. 목록에서 면적이나 가격으로 필터링하고, 분양가 범위를 보여 주는 값은 이 테이블을 집계해서 만듭니다. 그리고 앞에서 말씀드린 NULL과 0 구분이 바로 여기 특별공급 유형별 컬럼에 적용됩니다.

### 헷갈리기 쉬운 점

• notice에는 접수 상태 컬럼이 없습니다. 마감 여부는 날짜 비교입니다.

• 분양가 범위, 전용면적 범위는 컬럼이 아니라 notice_unit 집계입니다. 목록이 느려지면 뷰나 머티리얼라이즈드 뷰로 올린다는 것이 문서의 메모입니다.

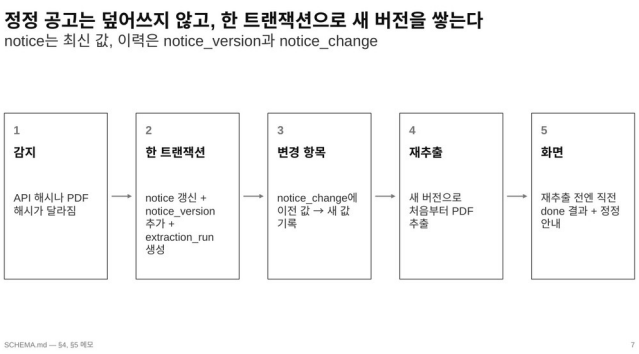
• 세대수 검증(주택형 합계와 총 세대수 비교)은 운영 로그에만 남기고 제약으로 막지 않습니다.

**해석 시 주의 (조건부 결과·불확실한 부분)**
is_regulated(규제 지역 여부)는 의미를 추정해서 만든 컬럼이라, 결정 D-03이 끝날 때까지 화면에 쓰지 않는다고 문서에 적혀 있습니다. 질문이 나오면 '현재는 저장만 하고 사용하지 않는다'고 답하세요.

**다음 슬라이드로 넘어가는 말**
그런데 공고는 한 번 올라오고 끝이 아닙니다. 정정이 올라오면 어떻게 되는지 보겠습니다.

# 슬라이드 7. 정정 공고는 덮어쓰지 않고, 한 트랜잭션으로 새 버전을 쌓는다

약 2분 · 출처: SCHEMA.md — §4, §5 메모




**핵심 메시지:** notice는 최신 값, 이력은 notice_version과 notice_change

### 먼저 알아야 할 배경지식

정정 공고는 청약홈에 같은 공고 번호로 내용이 바뀌어 올라오는 경우입니다. PDF 파일명에 '(2026.10.19.정정)' 같은 표시가 붙기도 합니다.

### 화면에서 먼저 가리킬 부분

맨 왼쪽 '감지'에서 출발해 화살표를 따라가며 두 번째 '한 트랜잭션' 상자에서 잠깐 멈춥니다.

### 발표 대본

정정 공고를 어떻게 다루는지가 이 스키마의 핵심 중 하나입니다. 원칙은 덮어쓰지 않고 버전을 쌓는다는 것입니다.

흐름을 따라가 보겠습니다. 먼저 수집할 때 API 원문의 해시나 공고 PDF의 해시가 직전과 달라지면 정정으로 판단합니다. 그러면 A가 한 트랜잭션 안에서 세 가지를 같이 합니다. notice의 최신 값을 갱신하고, notice_version에 새 버전 행을 추가하고, 그 버전의 추출 작업인 extraction_run도 만듭니다. 한 트랜잭션으로 묶는 이유는 버전만 생기고 추출 작업이 빠지는 어정쩡한 상태를 막기 위해서입니다.

notice_version에는 API 해시, PDF 해시, PDF 파일명, 저장 경로, 그리고 정정이 API에서 감지됐는지 PDF에서 감지됐는지가 들어갑니다. 그리고 notice_change에는 바뀐 항목의 이전 값과 새 값이 들어갑니다. 예를 들어 접수 종료일이 10월 27일에서 28일로 바뀌었다면 항목, 이전 값, 새 값이 한 행이고, 화면의 정정 블록과 정정 메일이 이 행을 읽어서 보여 줍니다. 바뀐 항목을 못 뽑았다면 notice_change 행은 만들지 않고 버전만 만들고, 메일에는 '공고문이 새로 올라왔어요'라고만 안내합니다.

마지막으로, 정정 직후에는 새 버전의 추출이 아직 끝나지 않았습니다. 이때 B는 직전 버전에서 가장 최근에 성공한 추출 결과를 보여 주고 정정 안내문을 함께 띄웁니다. 그리고 새 버전은 이전 근거를 재사용하지 않고 처음부터 다시 추출합니다.

### 숫자로 확인하기

예시: 공고123이 v1이었다가 10월 19일 정정되면 notice.current_version은 2가 되고 notice_version에는 (공고123, 1)과 (공고123, 2)가 모두 남습니다. 접수 종료일 변경은 notice_change에 (공고123, v2, 접수 종료일, 10/27, 10/28)로 기록됩니다.

**헷갈리기 쉬운 점**

• B가 보여 줄 추출 결과는 current_version 자체가 아니라 'current_version 이하에서 가장 최근에 done인 버전'입니다. current_version의 extraction_run이 pending일 수 있기 때문입니다.

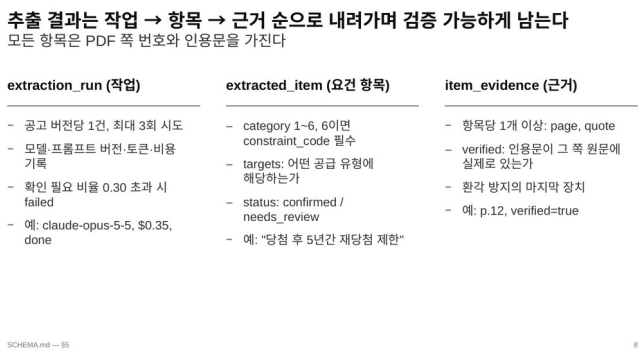
• 정정되면 항목을 버전별로 따로 두기 때문에 이전 버전의 근거를 가져다 쓰지 않습니다.

**다음 슬라이드로 넘어가는 말**

그럼 이 extraction_run 아래에 달리는 추출 결과 테이블들을 보겠습니다.

# 슬라이드 8. 추출 결과는 작업 → 항목 → 근거 순으로 내려가며 검증 가능하게 남는다

약 2.5분 · 출처: SCHEMA.md — §5




**핵심 메시지:** 모든 항목은 PDF 쪽 번호와 인용문을 가진다

### 먼저 알아야 할 배경지식

추출은 LLM이 공고 PDF를 읽고 청약 요건을 항목으로 뽑는 작업입니다. LLM은 없는 문장을 만들어 낼 수 있으므로, 뽑은 항목이 PDF 어디에서 나왔는지 확인할 수 있어야 합니다.

### 화면에서 먼저 가리킬 부분

맨 오른쪽 item_evidence의 'verified'를 먼저 짚고, 왼쪽에서 오른쪽으로 내려오는 구조를 설명합니다.

### 발표 대본

추출 영역은 세 단계로 내려갑니다. 작업, 항목, 근거입니다.

왼쪽 extraction_run은 공고 버전 하나당 한 번의 추출 작업입니다. 작업 큐 역할도 하고 비용 기록도 합니다. 실패하면 최초, 1시간 뒤, 6시간 뒤로 최대 3회까지 재시도하고, 몇 번째 시도인지는 attempt에 있습니다. 어떤 모델과 어떤 프롬프트 버전으로 돌렸는지, 입력과 출력 토큰이 얼마였고 비용이 얼마인지도 남깁니다. 그리고 '확인 필요'로 표시된 항목의 비율이 0.30을 넘으면 이 작업은 실패로 처리합니다. 결과를 믿기 어렵다고 보는 기준입니다.

가운데 extracted_item은 뽑아낸 요건 하나하나입니다. 분류는 1번부터 6번까지이고, 6번 제약 분류일 때만 재당첨 제한 같은 세부 코드가 반드시 들어가야 합니다. 이건 CHECK 제약으로 강제됩니다. targets는 이 요건이 전체에 적용되는지, 특정 공급 유형에만 적용되는지를 나타내고, status는 확실하다고 본 confirmed와 사람이 확인해야 하는 needs_review를 구분합니다. 예를 들어 '당첨 후 5년간 재당첨 제한'이라는 문장이 한 행입니다.

오른쪽 item_evidence가 가장 중요합니다. 항목마다 PDF 쪽 번호와 인용 문구를 최소 하나 붙이고, verified는 그 인용문이 해당 쪽 원문에 정말 문자열로 있는지 검사한 결과입니다. 즉 LLM이 지어낸 근거는 verified가 false로 남습니다. 화면에서는 이 인용문을 보여 줘서 사용자가 원문을 직접 확인할 수 있게 합니다.

### 헷갈리기 쉬운 점

• constraint_code 값 6개(rewin_limit 등)는 임시 이름입니다. SCREENS.md 블록 B와 맞춰 확정해야 합니다(미정 사항 1번).

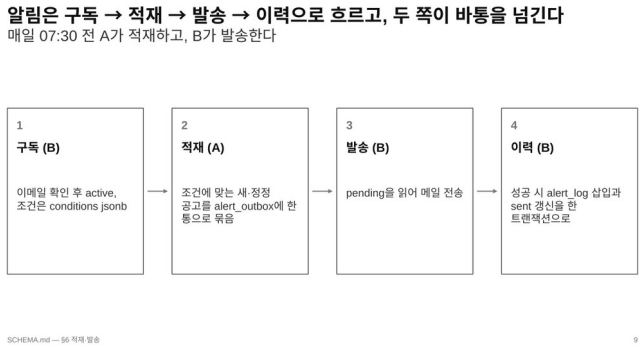
• summary_status는 notice에 있는 캐시 컬럼입니다. A가 추출이 끝날 때 최신 extraction_run을 보고 같이 갱신합니다.

**해석 시 주의 (조건부 결과·불확실한 부분)**
문서는 '확인 필요 비율 0.30 초과 시 failed'와 '화면은 verified 여부를 어떻게 표시하는가'를 따로 정의하지 않습니다. 후자는 화면 문서를 참고해야 합니다.

**다음 슬라이드로 넘어가는 말**
추출까지 끝났으면 사용자에게 알려 주는 알림 영역으로 넘어가겠습니다.

# 슬라이드 9. 알림은 구독 → 적재 → 발송 → 이력으로 흐르고, 두 쪽이 바통을 넘긴다

약 1.5분 · 출처: SCHEMA.md — §6 적재·발송




**핵심 메시지:** 매일 07:30 전 A가 적재하고, B가 발송한다

### 화면에서 먼저 가리킬 부분

맨 왼쪽 '구독 (B)'에서 시작해 두 번째 '적재 (A)' 상자에서 A로 넘어가고 세 번째 상자에서 다시 B로 돌아온다는 것을 손으로 따라가며 보여 줍니다.

### 발표 대본

알림은 네 단계로 흐릅니다. 사용자가 이메일을 입력해 구독을 신청하고 확인 메일의 링크를 누르면 구독이 active가 됩니다. 이때 필터 조건은 conditions라는 jsonb에 저장됩니다.

매일 아침 7시 30분 직전에 A가 모든 active 구독마다 그 조건에 맞는 새 공고와 정정 공고를 찾습니다. 이미 보낸 것은 alert_log를 보고 뺍니다. 보낼 게 남아 있으면 구독당 하루 한 통이 되도록 alert_outbox 한 행과, 그 메일에 담길 공고 목록인 alert_outbox_notice 여러 행을 한 트랜잭션으로 넣습니다. 이게 바통의 첫 번째 전달입니다.

그 다음 B가 pending 상태의 outbox를 읽어서 메일을 렌더링하고 발송합니다. 성공하면 alert_log에 이력을 넣고 outbox 상태를 sent로 바꾸는데, 이 둘을 한 트랜잭션으로 묶습니다. 하나만 성공해서 같은 메일이 두 번 나가거나 이력 없이 나가는 일을 막으려는 것입니다.

### 헷갈리기 쉬운 점

- 필터 조건 매칭은 A가 Python으로, 화면 필터는 B가 구현합니다. 같은 의미를 두 번 구현하므로 conditions 키 이름과 의미를 SCREENS.md에 표로 못 박아야 합니다(미정 사항 3번).

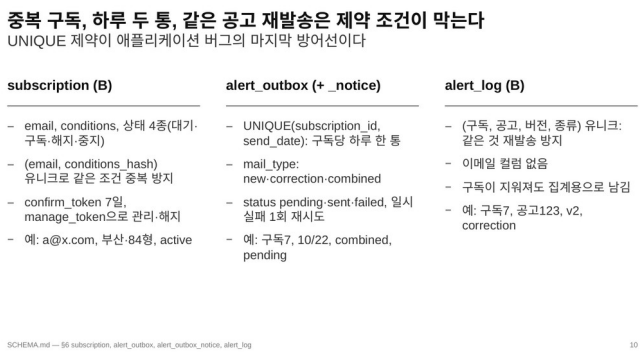
- 새 공고와 정정이 같은 날 겹치면 mail_type이 combined인 합본 한 통으로 나갑니다.

### 다음 슬라이드로 넘어가는 말

각 테이블이 이 흐름에서 어떤 제약을 갖는지 하나씩 보겠습니다.

# 슬라이드 10. 중복 구독, 하루 두 통, 같은 공고 재발송은 제약 조건이 막는다

약 2분 · 출처: SCHEMA.md — §6 subscription, alert_outbox, alert_outbox_notice, alert_log




**핵심 메시지:** UNIQUE 제약이 애플리케이션 버그의 마지막 방어선이다

### 화면에서 먼저 가리킬 부분

각 열의 두 번째 줄에 있는 UNIQUE를 차례로 짚습니다.

### 발표 대본

알림 테이블들의 공통점은 유니크 제약으로 중복을 막는다는 것입니다.

왼쪽 subscription은 구독 한 건입니다. 이메일 주소와 필터 조건, 상태가 있고 상태는 확인 대기, 구독 중, 해지, 발송 중지 네 가지입니다. 조건을 정렬해서 해시한 conditions_hash와 이메일의 조합에 유니크 인덱스가 걸려 있어서, 같은 사람이 같은 조건으로 두 번 구독하는 걸 막습니다. 확인 토큰은 7일 유효하고, 관리·해지용 링크는 별도의 manage_token을 씁니다. 이메일당 구독은 3개로 제한됩니다.

가운데 alert_outbox는 구독 ID와 발송 예정일에 유니크가 걸려 있습니다. 그래서 구독당 하루 한 통만 만들어집니다. 새 공고와 정정이 같은 날 겹치면 두 통이 아니라 합본 한 통입니다. alert_outbox_notice는 그 한 통에 담길 공고와 버전, 종류를 담는 자식 테이블입니다. 발송이 일시적으로 실패하면 한 번 재시도합니다.

오른쪽 alert_log는 보낸 이력입니다. 구독, 공고, 버전, 종류의 조합이 유니크라서 같은 정정 공고를 같은 사람에게 두 번 보내지 않습니다. 눈여겨볼 점은 이메일 컬럼이 없다는 것입니다. 구독이 삭제되면 구독 ID만 NULL로 바뀌고 이력은 집계용으로 남습니다.

### 축·범례·색상·화살표·박스의 의미

각 열의 마지막 줄 '예:'는 설명용 예시이며 실제 데이터가 아닙니다.

### 헷갈리기 쉬운 점

• alert_log의 유니크는 subscription_id가 NULL이 아닌 행에만 적용됩니다. 구독이 지워져 NULL이 된 이력은 중복 검사 대상이 아닙니다.

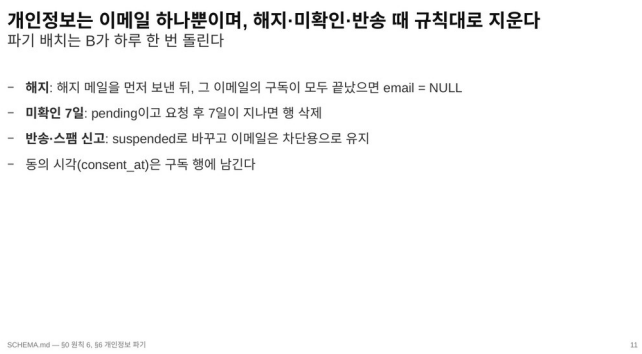
• subscription의 conditions에는 접수 상태와 검색어를 넣지 않습니다. 문서는 이유를 적지 않았습니다. 제 해석으로는 접수 상태가 시간이 지나면 바뀌는 값이라 조건에 넣을 수 없기 때문입니다.

### 다음 슬라이드로 넘어가는 말

마지막 관문은 개인정보입니다. 이메일을 언제 어떻게 지우는지 보겠습니다.

# 슬라이드 11. 개인정보는 이메일 하나뿐이며, 해지·미확인·반송 때 규칙대로 지운다

약 1.5분 · 출처: SCHEMA.md — §0 원칙 6, §6 개인정보 파기




**핵심 메시지:** 파기 배치는 B가 하루 한 번 돌린다

**화면에서 먼저 가리킬 부분**
세 줄을 위에서 아래로 짚되, 세 번째 줄 '이메일은 차단용으로 유지'에서 한 번 멈춥니다.

**발표 대본**
개인정보 규칙은 간단합니다. 우리가 가진 개인정보는 subscription의 이메일 주소 하나뿐이고, 필요 없어지면 바로 지웁니다.

첫째, 해지입니다. 사용자가 해지하면 해지 시각을 기록하고, 해지 확인 메일을 먼저 보냅니다. 그 다음 그 이메일의 모든 구독이 확인 대기나 구독 중이 아니게 되었을 때 이메일을 NULL로 지웁니다. 순서가 중요합니다. 지운 뒤에는 메일을 보낼 수 없으니까요.

둘째, 확인을 안 한 경우입니다. 구독 신청 후 확인 링크를 7일 안에 누르지 않으면 그 행을 삭제합니다.

셋째, 반송이나 스팸 신고가 들어온 경우입니다. 이건 이메일을 지우지 않고 오히려 유지하면서 상태를 발송 중지로 바꿉니다. 지워 버리면 같은 주소가 다시 구독을 시도했을 때 막을 방법이 없기 때문입니다. 재구독하면 확인 메일부터 다시 시작합니다.

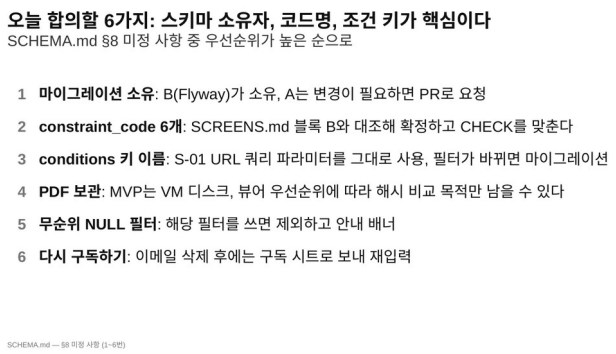
이 모든 파기는 B가 하루에 한 번 배치로 돌립니다.

**해석 시 주의 (조건부 결과·불확실한 부분)**
문서는 해지 후 이메일이 지워진 구독에서 [다시 구독하기] 버튼을 누르면 어떻게 동작하는지를 미정으로 둡니다. 제안은 조건이 채워진 구독 시트로 보내 이메일을 다시 입력받는 것입니다(미정 사항 4번).

**다음 슬라이드로 넘어가는 말**
마지막으로 오늘 합의가 필요한 미정 사항을 정리하겠습니다.

# 슬라이드 12. 오늘 합의할 6가지: 스키마 소유자, 코드명, 조건 키가 핵심이다

약 1.5분 · 출처: SCHEMA.md — §8 미정 사항 (1~6번)




**핵심 메시지:** SCHEMA.md §8 미정 사항 중 우선순위가 높은 순으로

### 발표 대본

정리하겠습니다. 이 스키마의 핵심은 쓰는 쪽을 테이블마다 하나로 정하고, 정정은 버전으로 쌓고, NULL과 0을 구분하고, 중복은 제약으로 막는다는 네 가지입니다.

그리고 합의가 필요한 사항이 문서 마지막에 여섯 가지 있습니다. 가장 먼저 정해야 하는 것은 마이그레이션 소유자입니다. 제안은 B가 Flyway로 스키마를 소유하고, A는 읽고 쓰기만 하며 필요한 변경은 PR로 요청하는 것입니다. 두 번째는 constraint_code 여섯 개의 이름입니다. 지금 값은 임시여서 화면 문서와 대조해 확정해야 합니다. 세 번째는 구독 조건 conditions의 키 이름인데, 화면 필터의 URL 쿼리 이름을 그대로 쓰자는 제안입니다. 이건 A와 B가 같은 의미를 각자 구현하므로 표로 못 박아야 합니다.

나머지 세 가지, PDF 보관 방식, 무순위 공고의 NULL 필터 동작, 이메일 삭제 후 다시 구독하기 동작은 제안이 적혀 있으니 동의하는지만 확인하면 됩니다. 이 여섯 가지에 대해 의견 주시면 이 문서를 확정 상태로 바꾸겠습니다.

### 헷갈리기 쉬운 점

- 여섯 항목의 우선순위 순서는 이 발표를 위해 제가 정한 것이며, 문서의 §8은 번호 순서로만 나열합니다.

## 부록 A. 용어 정리

| 용어 | 뜻 |
|---|---|
| house_manage_no | 청약홈의 주택관리번호. 공고 하나를 식별하는 기본키 |
| raw / raw_hash | API 응답 원문(jsonb)과 그 해시. 정정 감지에 쓴다 |
| jsonb | PostgreSQL의 JSON 저장 타입. 조건 매칭과 원문 보관에 쓴다 |
| partial index (부분 인덱스) | WHERE 조건에 맞는 행만 인덱싱. 예: status='success'인 수집만 |
| outbox | 발송할 메일을 먼저 DB에 쌓아 두는 대기열 패턴 |
| NULL vs 0 | NULL=API에 정보 없음, 0=실제로 0세대 |

## 부록 B. 코드 사항 메모: 이름과 정의 확인

• notice.rcept_start·rcept_end는 수집 때 규칙(P-01)으로 계산해 저장합니다. 규칙이 바뀌면 notice_schedule에서 다시 계산할 수 있습니다.

• pg_trgm 확장이 필요합니다(notice_name_trgm 인덱스, 공고 이름 검색).

• subscription의 CHECK 제약: 해지 상태가 아니면 email이 NULL일 수 없으나 suspended는 예외입니다.

## 출처 목록

• 원본: 청담 DB 스키마 (docs/spec/SCHEMA.md, 2026-10-21 초안). 모든 DDL·규칙·메모는 이 문서에서 가져왔으며, 예시 값은 설명용으로 만든 것이다.